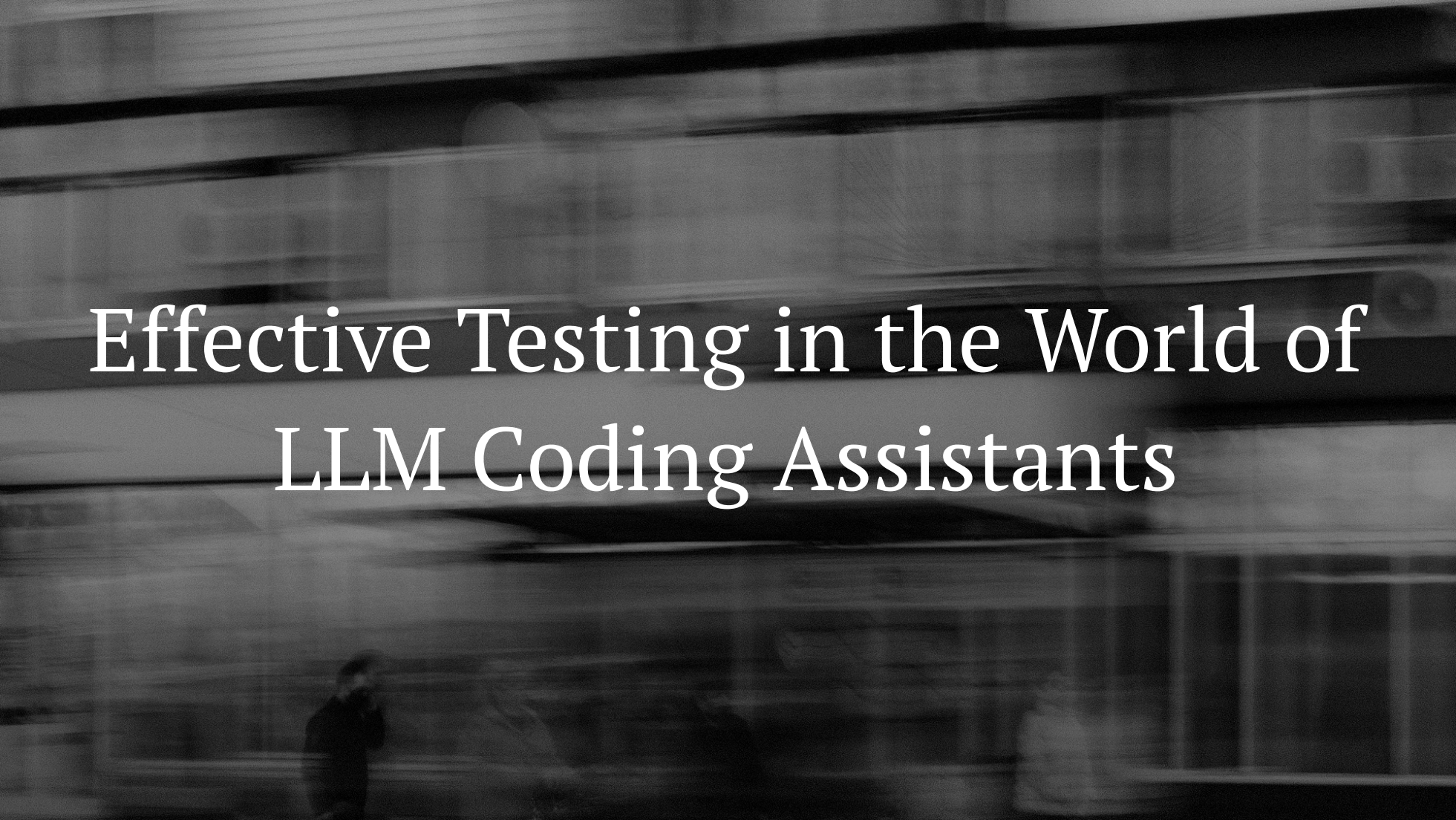




Effective Testing in the World of LLM Coding Assistants

Why Spend So Much Effort on
Testing? 🤔

Can You Spot The Bug?

```
def calculate_efficiency(completed, total):  
    return completed / total
```

How About Now?

```
def calculate_efficiency(completed: int, total: PositiveInt):  
    return completed / total
```



**It's a
Trap!**

It Depends!

```
from pydantic import PositiveInt
```

```
class PositiveInt:
    def __init__(self, value: int):
        if value <= 0:
            raise ValueError(f"{value} must be > 0")
        self._value = value

    def __rtruediv__(self, other: int) -> float:
        return other / self._value
```

```
def calculate_efficiency(completed: int, total: PositiveInt) -> float:
    return completed / total
```

```
# This crashes! 💥
calculate_efficiency(5, 0)
# ZeroDivisionError
```

```
# Can't even create it! 🛡️
calculate_efficiency(5, PositiveInt(0))
# ValueError
```

BUT WAIT

THERE'S VORE

Python Magic

```
from pydantic import PositiveInt

def patched_rtruediv(self, other):
    if self == 0:
        return 0
    return other / int(self)

PositiveInt.__rtruediv__ = patched_rtruediv

def calculate_efficiency(completed: int, total: PositiveInt):
    return completed / total
```

Turtles All The Way Down



Python has no bedrock of trust. **LLMs** must guess based on signals Python lets you lie about.

You could keep going forever - there's no solid foundation.

How to write good tests?

Don't Test the Framework

Test YOUR code, not Python or Pydantic

The Code:

```
@dataclass
class User:
    email: str
    age: int
    # Has __post_init__ with validation & normalization logic
```

Testing Python Works

```
def test_user_init():
    user = User("alice@example.com", 30)
    assert user.email == "alice@example.com"
    assert user.age == 30
```

Problem: Tests that dataclass initialization works, not YOUR logic!

Testing YOUR Logic

```
def test_normalizes_email():
    user = User(" ALICE@EXAMPLE.COM ", 30)
    assert user.email == "alice@example.com"

def test_rejects_negative_age():
    with pytest.raises(ValueError):
        User("alice@example.com", -5)
```

Better: Tests YOUR validation & normalization logic!

Focus on Behavior

When adding tests to existing code

The Code:

```
def load_config(path):  
    with open(path) as f:  
        return json.load(f)
```

✗ Brittle: Tests Implementation

Prompt: "Add tests for this function"

```
def test_load_config():  
    with patch('builtins.open',  
               mock_open(read_data='...')):  
        result = load_config('config.json')  
    assert result == {'key': 'value'}
```

Problem: Refactor to use `Path.read_text()` ? Test **breaks!** 💥

✓ Robust: Tests Behavior

Prompt: "Add tests that verify behavior, not implementation"

```
def test_load_config(tmp_path):  
    config_file = tmp_path / "config.json"  
    config_file.write_text('{"key": "value"}')  
  
    result = load_config(config_file)  
  
    assert result == {'key': 'value'}
```

Benefit: Refactor freely - test still passes! ✓

Beware Over-Mocking

The Golden Rule: Mock the boundaries, test the logic

The Code:

```
def process_user(user_id: int) -> dict:
    user = db.get_user(user_id) # External dependency
    if not validate_email(user['email']): # YOUR logic
        raise ValueError("Invalid email")
    return user
```

Testing Your Mocks

```
def test_process_user(mockeer):
    # Mock EVERYTHING including YOUR logic
    mock_db = mockeer.patch('db.get_user')
    mock_validator = mockeer.patch('validate_email')

    mock_db.return_value = {'email': 'test@example.com'}
    mock_validator.return_value = True

    result = process_user(1)
    assert result == {'email': 'test@example.com'}
```

Problem: `validate_email` could be broken and this passes! 💣

Mock External, Test Internal

```
def test_validates_email(mockeer):
    # Mock ONLY external dependency
    mock_db = mockeer.patch('db.get_user')
    mock_db.return_value = {'email': 'INVALID-EMAIL'}

    # YOUR validation runs for real
    with pytest.raises(ValueError, match="Invalid"):
        process_user(1)
```

Better: Mock database, test YOUR validation logic!

Single Responsibility Principle

One test, one behavior

❌ The "Mega Test"

```
def test_user_functionality():
    # Test 1: Creation
    user = User(email="test@example.com", age=25)
    assert user.email == "test@example.com"

    # Test 2: Validation
    with pytest.raises(ValueError):
        User(email="invalid", age=25)

    # Test 3: Edge case
    with pytest.raises(ValueError):
        User(email="test@example.com", age=-1)

    # Test 4: Transformation
    user2 = User(email=" ALICE@EXAMPLE.COM ", age=30)
    assert user2.email == "alice@example.com"
```

When this fails, what broke? Creation? Validation? Edge case?

Transformation?

✅ Focused Tests

```
def test_normalizes_email_to_lowercase():
    user = User(email="ALICE@EXAMPLE.COM", age=25)
    assert user.email == "alice@example.com"

def test_rejects_invalid_email_format():
    with pytest.raises(ValueError, match="Invalid"):
        User(email="invalid", age=25)

def test_rejects_negative_age():
    with pytest.raises(ValueError, match="negative"):
        User(email="test@example.com", age=-1)
```

When `test_rejects_negative_age` **fails:** Immediately know what broke!

Use Minimal Test Data

Does having 100 items prove `max()` works better than 2 items?

No.

✗ Unnecessarily Large

```
def test_calculate_max():
    objects = [
        Item(value=10), Item(value=25),
        Item(value=33), Item(value=47),
        # ... 94 more items ...
        Item(value=167)
    ]

    assert stats['max'] == 167
```

Can you verify 167 is the max? 🤔

✓ Minimal and Sufficient

```
def test_calculate_max():
    objects = [
        Item(value=10),
        Item(value=30)
    ]

    assert stats['max'] == 30
```

2 items proves `max()` picks the larger one

Can you verify $30 > 10$? ✓ Yes!

Use the **minimum data that proves the behavior** (1 item = not enough, 2 items = sufficient, 100 items = overkill)

Red - Green - Refactor

When fixing bugs with LLMs

The Trap

Prompt: "Fix the bug in `add(a, b)` "

LLM generates **both** fix and test:

```
def add(a, b):  
-   return 3 # Bug!  
+   return a + b # Fixed!
```

```
def test_add():  
    assert add(1, 2) == 3 #  Passes!
```

Problem: Test never failed. Did it catch the bug?

The Solution

Prompt: "Write a failing test, then fix it"

 **Red:** Write test first

```
def test_add_returns_sum():  
    assert add(2, 3) == 5 #  Fails!
```

 **Green:** Fix the code

```
def add(a, b):  
    return a + b #  Now passes!
```

Benefit: Proves the bug existed AND is fixed

Practical Prompting Strategies

✗ Instead of: **"Write tests for this function"**

✓ **"Test my validation logic, not the framework"**

Avoids testing Python/Pydantic

✓ **"Test behavior, not implementation"**

Prevents brittle tests & over-mocking

✓ **"One test per behavior"**

Clear failures, focused tests

✓ **"Write a failing test first"**

Ensures Red-Green-Refactor cycle

✓ **"Use minimal test data"**

Just enough to prove the behavior

✓ **"Mock boundaries, test my logic"**

External deps only, not business logic

Why Spend So Much Effort on Testing?

Python + LLMs = Double Uncertainty

Tests are your reality check

They verify what the LLM *assumed* matches what actually *runs*

Not optional. Not overhead. **Essential verification.**